ECE551 / MIDTERM REFERENCE

Language • arithmetic • hardware • simulation | Q# refers to the Spring 2020 practice exam

TYPES / STRUCTURAL / OPERATORS [Q1,5,10]

Hardware first: assign/complete always_comb → gates/muxes; incomplete combinational assignment → latch; edge-triggered block → FF. Separate blocks run concurrently. A reg declaration alone does not imply a register.

Legacy: procedural LHS → reg; continuous/gate output → wire. **SV:** logic for a single driver; wire for resolved/multiple drivers. 0/1/x/z = low/high/unknown/undriven. Uninitialized storage is x. Equal-strength opposing drivers → x; stronger wins; z + weak drive → weak drive's value. Strength: supply > strong > pull > weak > highz.

```
and g(y,a,b); // output first
bufif1 g(y,a,en); // en=0 -> z
adder #(.WIDTH(16)) u(.a(A),.sum(S));
```

Parameter: overridable per instance. localparam: fixed local constant. defparam: legacy hierarchical override statement; avoid. Backtick define: preprocessor macro, not a per-instance parameter. Backtick default_nettype none catches implicit one-bit nets/typos.

Binary vs unary: a&b bitwise; a&&b one-bit truth; &v all bits1; |v any bit1; ^v odd parity. ~v bitwise invert; !v one-bit zero test. ~|cnt → zero; &cnt → all ones/full. ==/!= may yield x; ===/!== compare x/z literally (TB). 0&x=0; 1|x=1. ?: infers mux; x select merges equal branch bits.

Q1 quick: A5 & 0F = 05; |05 = 1. `#(1:2:3)` = min:typ:max for both edges; `#(2,1)` = rise,fall. RTL delays are simulation models, ignored by synthesis. Timescale sets units/precision.

WIDTH / SIGN / SHIFT RECIPES [Q5]

N-bit unsigned: 0..2^N−1. Signed: −2^(N−1)..2^(N−1)−1. Use sized signed constants: 5'sd9. `$signed` changes sign interpretation, not width. Mixed signed/unsigned often becomes unsigned. Concatenations/part-selects are unsigned; cast when used in signed math/comparisons. Extend **before** arithmetic; keep carry/product bits.

```
assign vec16 = SE ?
  {{8{vec8[7]}},vec8} : {8'h00,vec8};
assign {co,S} = {1'b0,A}+{1'b0,B}+cin;
assign rotR = {v[0],v[15:1]};
assign shL = {v[14:0],in_bit};
```

N+N sum needs N+1 bits; N×M full product needs N+M. >> zero-fills; >>> sign-fills only for signed left operand. `$signed(4'b1001)>>>2` = 1110 (−2). Dropping k LSBs / signed >>>k rounds down: −65>>>6=−2; signed −65/64=−1. Negating the minimum signed value needs widening. `'0/'1` fill the destination.

OVERFLOW + SATURATION [HW2, EX04–07]

Signed add overflow: same operand signs, changed result sign. Subtract: different operand signs, changed A sign. Carry is separate. 7F+01→80: OV1, carry0; FF+01→00: OV0, carry1.

```
ov_add = ~(A[N-1]^B[N-1]) &
          (S[N-1]^A[N-1]);
ov_sub =  (A[N-1]^B[N-1]) &
          (S[N-1]^A[N-1]);
// A-B = A + ~B + 1
```

Unsigned W→N: discarded bits nonzero → all-ones rail. Signed W→N: bits [W−1:N−1] must all equal sign. Check includes **new sign bit N−1**. Saturate clips; truncate wraps.

```
// unsigned 16->10
usat = |u[15:10] ? 10'h3FF : u[9:0];
// signed 16->10: [-512,511]
ssat = (!s[15] && |s[14:9]) ? 10'h1FF:
       (s[15] && ~&s[14:9]) ? 10'h200:
       s[9:0];
// signed 10->7: test d[8:6], sign d[9]
// rails 7'h3F (+63), 7'h40 (-64)
```

Test min−1/min/min+1/−1/0/max−1/max/max+1. Ex04 trap: 80−01→7F but OV must be1. 16→12 signed rails 7FF/800; test bits [14:11], sign bit15.

CASE / COMBINATIONAL DEFAULTS

```
always_comb begin
 y=0; nxt_state=state; // assigned always
 case(state)
   S0: if(go) begin y=1; nxt_state=S1; end
   S1: nxt_state=S0;
   default: nxt_state=S0;
 endcase
end
```

case exact x/z match. casez ignores z/? in expression AND items; casex also ignores x (can hide bugs). case inside wildcards in items. First match wins. unique declares exclusivity; priority declares priority/expected match. Defaults prevent unintended hold and recover illegal states.

DRAW THE HARDWARE [Q2,8,9,14]

Method: trigger → state elements → each D/hold equation → arithmetic/gates → muxes → reset and enable labels. Missing clocked branch means FF hold (feedback mux); missing combinational branch means latch. if/else-if means earlier condition wins.

Q2 casez latch: 01: out=~a; z0: out=b; no default. Table 00→0,01→1,10→0,11→hold. **Draw latch D=b, EN=~(a&b).** Procedural out is reg; outer begin/end unnecessary (case is one statement). always_comb can warn about inferred latch; cannot fix missing assignments.

Q9 partial reset: posedge block: if(!rst_n) q1<=0; else {q1<=d; q2<=q1;}. Draw **two DFFs** with input muxes:

```
D1 = rst_n ? d  : 1'b0;
D2 = rst_n ? q1 : q2; // hold on reset
```

Reset is synchronous (rst_n absent from trigger). q2 is not reset; do not draw a simple unconditional pipeline. All NBA RHSs see pre-edge q1.

STORAGE TEMPLATES [EX08 / HW2]

```
// sync high reset
always_ff @(posedge clk)
 if(rst) q <= 0;
 else q <= d;

// async low reset + sync enable
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) q <= 0;
 else if(en) q <= d; // else hold

// reset-dominant SR / sticky flag
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) flag <= 0;
 else if(clear) flag <= 0;
 else if(set) flag <= 1;
// no last else: keeps previous flag

// actual level-sensitive latch
always_latch if(en) q <= d;
```

Q14 fix: with async-reset sensitivity, never combine synchronous clear into `if(!rst_n | clear)`. Separate reset, then `else if(clear)`, then set. A missing final else in a clocked block holds a FF; it is not a latch.

Ex08 trap: always @(clk) if(clk) q<=d misses d changes while clk high → faulty latch simulation. Use always_latch / @(*). always_ff expresses intent/restricts writers/timing; it does not insert a clock or guarantee every variable survives synthesis.

Q8 counter bug: bit_cnt=bit_cnt+1 in always_comb gives self-feedback, not a clock counter. Emit inc_bit; increment stored count in always_ff. A proper next-count equation is fine if assigned to a distinct signal then registered.

SYNC / EDGES / SHIFTERS [Q11,12]

Two FFs synchronize a single async bit; third stores old synchronized value. Reduces metastability propagation risk; not debouncing or coherent multibit transfer. Reset/preset stages equal to idle level (1 for UART/PB).

```
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) begin
  meta<=1; sync<=1; prev<=1;
 end else begin
  meta<=async_in; sync<=meta; prev<=sync;
 end
assign rise = sync & ~prev;
assign fall = ~sync & prev;
```

Q12 course expects wire for legacy assign output, reset/preset, and <= for FFs. Stages suggest async input, but do not logically prove it. Reverse blocking order can preserve this chain in one block; still use NBA to avoid inter-process races.

```
// Q11: oldest at bit7, newest bit0
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) sh<=8'h00;
 else sh<={sh[6:0],in_stream};
assign match=(sh==8'hAB);
```

Ex09 reset: async assertion; release via two negedge FFs (D1=1,D2=Q1), away from main posedge. PB release uses rising detector; idle PB is1.

Swap/LFSR: a<=b; b<=a swaps old values. XOR-feedback q<={q[1:0],q[2]^q[0]} uses old state; seed nonzero. Same-process NBAs to same LHS/time: last executed wins. Cross-process writers can race.

Quick drill: before edge q1=0,q2=1,d=1, with q1<=d; q2<=q1 → after edge 1,0. If reset branch only assigns q1, q2 keeps1. With en0 in combinational if(en)q=d, draw a latch; with en0 in posedge if(en)q<=d, draw a held FF.

SIMULATION / RACES [Q4,6,7,13]

Course queue: active (blocking, RHS evaluations, continuous assigns, display) → inactive (#0) → NBA LHS updates → monitor/strobe; repeat triggered activity until settled, then advance time. Full SV adds assertion/other regions.

=: evaluate/update before next statement. <=: evaluate now, update later in NBA. Within a begin/end order is fixed; independent blocks at same time have no general order. Race means multiple legal results; need not literally become x.

```
// Before edge q1=0,q2=1,d=1:
q1 <= d; q2 <= q1; // after NBA: 1,0
q1 = d;  q2 = q1;  // in this block: 1,1
```

Hand trace: write time / RHS sample / scheduled LHS update. A sampled delayed RHS is not reevaluated when its source changes.

Four delay forms at t:

`#5 q=d;` sample/update t+5; process waits.

`q=#5 d;` sample t, update t+5; process waits.

`q<=#5 d;` sample t, NBA update t+5; continue now.

`#5 q<=d;` wait, sample t+5, NBA update then.

```
// a=3,b=2,c=1 BEFORE clean event:
b <= a+a;      // b=6 now (NBA)
c <= #5 b+a;   // c=5 at +5
d <= #2 c+a;   // d=4 at +2
// Instead: b<=a+a; #5 c<=b+a;
// #2 d<=c+a; -> c=9 at +5,d=12 at +7
```

Q7 race: initial {a=0;b=1;} and initial {a=1;#5 b=0;} → at settled t5, a=0 OR1; b=0. Source order across blocks does not settle a.

Observe: display prints when reached (often old FF values); strobe prints settled values; monitor prints settled changes. stop pauses for interactive debug; finish exits batch; fatal fails/exits. #0 is inactive-region delay, not a robust race cure.

Q6 sensitivity: given stg1<=en?a^b:a|c; out<=stg1&c; use @(a,b,c,en,stg1) because NBA stg1 must retrigger. Prefer ordered blocking in always_comb. In combinational procedures include all read inputs and conditions; do not assume a self-written temporary will retrigger always_comb.

RING / GATE DELAYS [Q3,4]

```
wire n1,n2;
nand #5 g1(n1,EN,out);
not  #5 g2(n2,n1);
not  #5 g3(out,n2);
```

EN low initializes out=1 after settling. EN high enables 3 inversions. Half-period=sum delays=15; period30. Gate delays are inertial (short pulses filtered). Delayed continuous assigns also filter pulses. Delayed NBA can preserve each sampled transition (transport-style model).

```
// Blocking version does NOT self-oscillate:
always @(EN or out)
 out = #15 (EN ? ~out : 1'b1);
// Fix for this simulation model:
always @(EN or out)
 out <= #15 (EN ? ~out : 1'b1);
```

Blocking process is busy when it changes out and misses the event. NBA returns to event wait before update, so re-triggers. Hold EN low long enough first. This feedback-delay model is a simulation exercise.

Clock TB: initial clk=0; always #10 clk=~clk; at 1ns units gives 20ns period/50MHz. No event/delay in an infinite always loop can freeze time.

CUMMINGS RULES TO RECALL

Sequential FFs and latches: <=. Combinational procedures: =. A clocked update with arithmetic, e.g. cnt<=cnt+1, still uses <=. Do not mix =/<= in one block; no multiple procedural writers; no #0 assignments; use strobe for NBA outputs. Different reset/enables → separate FF groups.

Race-free TB: drive inputs at negedge for posedge DUT; check @(posedge clk); #1; with suitable timescale/clock period. Check combinational results only after settling. A valid waveform is evidence for that stimulus; a self-check and coverage make it stronger.

Delay drill: d1 at t0, d0 at t2; q<=#5 d at t0 → q1 at t5. q=#5 d also assigns1 at t5 but blocks its process. #5 q=d instead samples0 at t5.

ECE551 / MIDTERM REFERENCE

FSMs • UART/PWM/PID • verification • FIFO • timing | Q# refers to the Spring 2020 practice exam

FSM RECIPE + PRACTICE Q16 [18 PTS]

List state/data/counters/flags; map spec actions to load/clear/count/shift signals; map datapath comparisons/done to inputs. Moore outputs depend on state; Mealy on state+inputs. Registering outputs adds storage/latency. Combinational decode can glitch.

Q16 sequence: IDLE --start/wrt→ CHNNL --done→ WAIT --one cycle/wrt→ RESP --done/set_cc→ IDLE. CHNNL/RESP self-loop without done. WAIT explicitly inserts a full state cycle between accepted transactions. Underlying SPI done clears on new start.

```
typedef enum logic [1:0]
 {IDLE,CHNNL,WAIT,RESP} state_t;
state_t state,nxt_state;
logic wrt,set_cc;
always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) state<=IDLE;
 else state<=nxt_state;

always_comb begin
 nxt_state=state; wrt=0; set_cc=0;
 case(state)
  IDLE: if(strt_cnv) begin
   wrt=1; nxt_state=CHNNL;
  end
  CHNNL: if(done) nxt_state=WAIT;
  WAIT: begin wrt=1; nxt_state=RESP; end
  RESP: if(done) begin
   set_cc=1; nxt_state=IDLE;
  end
  default: nxt_state=IDLE;
 endcase
end

always_ff @(posedge clk or negedge rst_n)
 if(!rst_n) cnv_cmplt<=0;
 else if(strt_cnv) cnv_cmplt<=0;
 else if(set_cc) cnv_cmplt<=1;
// Sticky until new request; no else needed.
assign cmd={2'b00,chnnl,11'b0};
// SPI rd_data is 16-bit: use temp rd16
assign res=rd16[11:0];
```

Old key corrections: reset state to IDLE, not state<=state; use blocking nxt_state=state, not <= in always_comb; spell cnv_cmplt consistently; handle 16→12 output width explicitly. Reset arrow + all self-loops + output labels on bubble diagram earn clarity/partial credit.

FSM mistakes: no output defaults → latch; counter update in comb → feedback; no illegal-state default → poor recovery; done held from previous transfer → premature exit; repeated starts in wait states → duplicate operations. Distinguish one-cycle control pulses from sticky stored flags.

CURRENT STEERING FSM [HW3]

Three states: WAIT_RIDER, WAIT_BALANCE, STEERING. Defaults nxt=state, clr_tmr=en_steer=rider_off=0. Timer counts unless cleared.

WAIT_RIDER: rider_off=1, clr_tmr=1. sum_gt_min (with no sum_lt_min) → WAIT_BALANCE.

WAIT_BALANCE, priority order: (1) sum_lt_min → rider_off=1, clr_tmr=1, WAIT_RIDER. (2) diff_gt_1_4 → clr_tmr=1, stay. (3) tmr_full → STEERING. Else stay and count continuous balanced time.

STEERING, priority order: (1) sum_lt_min → rider_off=1, clr_tmr=1, WAIT_RIDER. (2) diff_gt_15_16 → clr_tmr=1, WAIT_BALANCE. Else en_steer=1. This code removes steering immediately on exit conditions. Mild >¼ imbalance alone does not disable already enabled steering.

Likely variation: require an input stable for N cycles before enabling an output. States OFF/QUALIFY/ON; clear timer in OFF and on bad input; enter ON only after good input persists until timeout. Restart timer on requalification, do not reuse old tmr_full. Preserve priority and hysteresis.

Timer math: W bits count 0..2^W−1; full reduction &. N-clock interval uses 0..N−1; width ceil(log2 N), if only those states are needed. To store count N inclusive use ceil(log2(N+1)). An extra wait state adds a clock cycle, not a #delay.

PWM / NON-OVERLAP [Q14; EX09]

N-bit free-running counter: period2^N clocks, f_pwm=f_clk/2^N. Simple cnt<duty gives duty high clocks. 11-bit at50MHz: 2048 clocks,40.96µs,24.414kHz. duty0→0%;2047→2047/2048 (not100%).

Current PWM11: NONOVERLAP=64=1.28µs. PWM1 clear if old cnt>=duty, else set if old cnt>=64. PWM2 clear if &cnt (2047), else set if old cnt>=duty+64. Clear beats set. Both FF outputs hold otherwise; no latch. Pre-edge count sets outputs; current post-edge count is incremented.

```
// Widen threshold; avoid 11-bit wrap!
logic [11:0] pwm2_start;
assign pwm2_start={1'b0,duty}+12'd64;
assign PWM_synch=~|cnt;
```

Steady high counts: PWM1=max(duty−64,0); PWM2=max(1983−duty,0). duty1024→960/959 clocks; duty65→1/1918. Update duty at PWM_synch. Never PWM1&&PWM2 (shoot-through). Blanking ignores switch-current spikes: strict 64<cnt<192 OR duty+64<cnt<duty+192. Strict windows select127 values, despite slide prose128; end window truncates.

UART 8N1 [EX11,12 / HW3]

Idle1, start0, D0..D7 **LSB first**, stop1. No clock wire; receiver samples locally. 50MHz/9600≈5208 clocks/bit; 13-bit timer **0..5207**. 4-bit counter for10 frame bits. Bit104.16µs; frame52080 clocks=1.0416ms.

```
// TX: 9 stored bits; ones provide stop
if(load) sh <= {tx_data,1'b0};
else if(shift) sh <= {1'b1,sh[8:1]};
assign TX=sh[0];
// shift each full baud, count bit0..9
// set_done at end of complete stop bit
```

TX IDLE accepts trmt; TRANSMIT counts/shifts. load clears bit/baud counters and sticky tx_done. Reset sets sh=9'h1FF so TX idle1. Captured byte unaffected by later tx_data changes; busy requests ignored.

RX: double-sync + falling-edge detect; start loads down-timer2603 (2604 clocks); each sample reload5207 (5208). Sample at centers: start at½ bit, then D0 at1½,...,stop9½. Ten shifts into9-bit register:

```
if(sample) sh <= {RX_sync,sh[8:1]};
assign rx_data=sh[7:0]; // stop is bit8
```

Start falls out; data lands in correct order. RX rdy at stop center, ~½ bit before TX done. rdy sticky until clr_rdy OR new start; clear wins over set. rx_data changes during next reception—consume while rdy1. Test 00/FF/A5/5A/01/80, back-to-back frames, clear/hold flags, reset midframe.

PID / SEGWAY MATH [EX06,07,10]

e=sat10(ptch); P=9e (15-bit); I=integrator>>>6 (15-bit); D=−(sign-extended ptch_rt>>>6) (13-bit); sum sign-extended16; PID=sat12(sum). Example ptch100,int640,rate128→900+10−2=908. D uses measured rate, not finite differences.

18-bit integrator: async reset OR synchronous rider_off →0; else if(vld && !signed_overflow) add e; otherwise hold. Overflow rejects update, not clamps. Power-off clears27-bit long_tmr; increment until &[26:19], then freeze. ss_tmr=long_tmr[26:19]. Saturates after255×2^19 clocks≈2.674s at50MHz. Use complete **HW3/PID.sv**; Ex10 copy lacks I_term assignment.

Soft-start base=(PID×$signed({1'b0,ss_tmr}))>>>8. Steering: clip pot [200,E00], subtract7FF, multiply3, >>>4. Left/right torque=base±steer if enabled (13 signed bits). Shape: |t|<42→4t; else t±168 outward. pwr_up0→0; sat12 each. too_fast if signed lft_spd>1536 OR rght_spd>1536 (positive threshold, not abs).

Arithmetic drill: sat10(+600)=1FF; sat10(−600)=200. PID256×ss128 /256→128, centered steering→0, shape adds168→both speeds296. At PID−1×ss128, dropping8 low bits gives−1, then small-torque shape gives−4.

FILES / SHARED DEFINITIONS [LECTURE 12]

Files: fopen(name,"r") returns0 on failure; fscanf returns matched-field count; fgetc use int for EOF−1; fgets line; readmemh/readmemb ASCII memory init; fdisplay line; fclose. `logic[7:0] mem[0:15]`=16 byte words. Package shares definitions; import pkg::*; backtick include inserts source.

SELF-CHECK / TIMEOUT / SV TB [Q13]

TB has no ports, instantiates DUT, initializes inputs, generates clock/stimulus, independently computes expected, checks and ends. Drive negedge/check after NBA. !== catches x/z rather than allowing unknown comparisons to hide failure.

```
// 4-bit A,B, one-bit cin: 512 cases
for(int i=0;i<512;i++) begin
 {A,B,cin}=i[8:0];
 exp={1'b0,A}+{1'b0,B}+cin; // exp[4:0]
 #1;
 assert({co,Sum}===exp)
  else $fatal(1,"mismatch");
end
$display("PASS"); $finish;
```

Use int iterators; 4-bit iterator wraps at15, never reaches16. Ex04 all A/B/SUB=131072 cases. Directed tests hit rails, priority overlaps, reset, hold, timer boundaries. Random tests need checks/coverage, not just random stimulus.

```
fork : wait_or_timeout
 begin wait(rdy); end // handles already1
 begin repeat(60000) @(posedge clk);
  $fatal(1,"timeout"); end
join_any
disable wait_or_timeout;
```

join waits all; join_any one; join_none parent continues. Kill leftover timeout. Task may consume time; function cannot #/@/wait. Live monitored arg needs **automatic task + ref logic**, not input snapshot. Generate/genvar elaborates fixed hardware; loop iterations are not clock cycles without event controls.

```
assert property (@(posedge clk)
 disable iff(!rst_n)
 req |-> ##[1:2] ack);
```

Each sampled req requires ack1–2 clocks later. |-> same-tick consequent; |=> next-tick. Concurrent assertions sample before NBA. Immediate assert runs when reached.

SYNC FIFO BOUNDARIES [EX13]

Power-of-2 depth D. ptr bits=clog2(D); count bits=clog2(D+1) (D16→4 vs5). full=(count==D); empty=(count==0). Decisions use **pre-edge** flags. No read on empty, no empty bypass.

```
do_rd = rd_en && !empty;
do_wr = wr_en && (!full || do_rd);
// clocked, after reset:
case({do_wr,do_rd})
 2'b10: count<=count+1'b1;
 2'b01: count<=count-1'b1;
 default: ; // hold
endcase
overflow <= wr_en && full && !do_rd;
underflow <= rd_en && empty;
```

Both enables at full: both succeed, countD, no overflow, read old head. Both at empty: only write, count1, underflow1, rd_data holds. Middle: both succeed, same count. Advance pointers only on successful op; registered rd_data holds otherwise. Reset ptr/count/data/errors0; memory validity follows count. Error flags are **one-cycle pulses**, assigned every cycle.

Five bug hunts: full atD−1; too-narrow count; rejecting full simultaneous write; sticky underflow; advancing read while empty. Test fill D then extra write; full read/write; drain; empty read then idle (pulse must clear); empty read/write then read known byte; multiple wraps/reset. Queue reference from spec avoids copying DUT bugs.

SDC / DESIGN CHOICE [Q1,15]

No skew/uncertainty: t_cq+t_comb+t_setup≤T. Internal comb budget=T−cq−setup. Input delay=external cq+external logic/wire. Input-to-FF budget=T−input_delay−setup. Hold: cq_min+comb_min≥hold (no skew). Max delay threatens setup; min delay threatens hold.

Q15:0.25+3.25=3.5ns input delay. T5,setup0.25→DUT logic1.25ns. Internal path with cq/setup0.25→4.5ns.

```
create_clock -name clk -period 5 clk
set_input_delay 3.5 -clock clk sig
```

Outputs need required timing relative to clock (**set_output_delay**) AND capacitive load (**set_load**). Q1f key circles load; full timing setup needs both. #delays do not constrain synthesis. Flow: spec→RTL→functional sim→synthesis/netlist→place/route→parasitics/timing→hardware. ASIC: volume/efficiency; FPGA: programmable fast development; MCU: software/peripherals; full custom: highest effort/control. Honda Q1 expects standard-cell ASIC.

Final scan: signs/widths; complete comb paths; FF holds; reset sensitivity; clear/set priority; pre-edge values; inclusive counts; pulses vs sticky flags; default-state recovery. Draw storage and feedback muxes explicitly.